



AVN

INSTITUTE OF
ENGINEERING & TECHNOLOGY
Accredited by NAAC & NBA

NBA
NATIONAL BOARD
FOR ACCREDITATION



DEPARTMENT OF CSE (DATA SCIENCE)

Augmented Reality-Based Navigation and Recognition System

BY TEAM-11

K.Laxman	235U1A6732
G.Srivani	235U1A6721
T.Deekshitha	235U1A6757
J.Madhumitha	235U1A6726

Under the Guidance of

Mr.A.Narender

ACADEMIC BATCH 2023-2027

ABSTRACT

- **Augmented Reality (AR)** overlays digital elements like arrows, directions, and labels on the real-world camera view.
- The system provides **real-time navigation** using a smartphone camera with **GPS and sensors**.
- It shows **visual routes directly on the screen**, making navigation easier than traditional map-based apps.
- The application can be used in **campuses, malls, airports, outdoor navigation, and smart cities**, improving the overall user experience.



INTRODUCTION

- **Concept & Overview:** The project uses **Augmented Reality (AR) with GPS** to create a smart navigation system that shows directions through the smartphone camera.
- **Technology Used:** It integrates **AR technology, GPS, mobile sensors, camera, and mapping services** for accurate location tracking.
- **Functionality & Features:** The system detects the user's location and displays **live arrows, path indicators, and destination details** with an easy interactive interface.
- **Goal & Benefits:** The main aim is to provide **real-time, accurate, and user-friendly navigation** that is easier to follow than traditional map-based systems.

LITERATURE REVIEW

Project	Year	Approach (Technology Used)	Merits	Demerits
AR Navigation using SLAM Algorithm	2023	Simultaneous Localization and Mapping (SLAM), Sensor Fusion, Smartphone Camera	Markerless tracking, High accuracy, Works in dynamic environments	High computational cost, Requires powerful hardware
AR-Based Pedestrian Navigation System	2022	GPS, IMU Sensors, Computer Vision	Real-time directional guidance, Easy mobile implementation, Improves user experience	GPS errors in urban/indoor areas, Battery consumption
AI-Enhanced AR Navigation System	2020	Artificial Intelligence, Deep Learning, Object Recognition, Sensor Fusion	Intelligent path prediction, Adaptive routing, Better object detection	Complex implementation, High processing and storage requirement

EXISTING SYSTEM



Uses: GPS, smartphone camera, and basic sensors for location tracking.



Depends: On internet connectivity and marker/map-based tracking systems.



Performs: Real-time overlay of navigation directions on the live camera view.



Requires: AR-supported smartphones with good processing capability.



Needs: Continuous GPS signal and sufficient battery backup.



Works: Efficiently in outdoor environments but has limitations indoors.



Response Speed: Moderately fast, but may slow down in weak signal or complex environments.

DISADVANTAGES OF EXISTING SYSTEM



Inaccurate GPS: Poor location accuracy in indoor or dense urban areas.



Internet Dependency: Requires continuous internet connection to function properly.



High Battery Usage: Constant camera and sensor usage drains battery quickly.



Device Limitation: Needs AR-supported and high-performance smartphones.



Environmental Sensitivity: Performance reduces in low light or bad weather conditions.



Marker Dependency: Marker-based systems require physical installation, limiting scalability.



Moderate Response Speed: May lag in complex or crowded environments.

PROPOSED SYSTEM

- **Real-Time AR Navigation:** Live camera view with AR arrows and markers for clear route guidance.
- **Smart Localization & Vision:** Uses GPS, sensors, and computer vision (SLAM) to detect roads, signs, and landmarks for accurate positioning.
- **Adaptive & Multi-Mode Navigation:** Dynamic path updates with pedestrian, driver, and indoor navigation modes.
- **User-Centric System:** Voice/gesture/touch interaction, accessibility features, personalized profiles, cloud map updates, and secure data handling on smartphones or AR glasses.

ADVANTAGES



- **Easy to Follow Directions:** Shows arrows and directions directly on the real-world view, so users can understand routes easily.
- **Better User Experience:** Interactive visual guidance makes navigation more engaging than traditional maps.
- **Reduces Confusion:** Real-time AR directions help users avoid wrong turns and navigate more accurately.
- **Useful in Many Places:** Can be used in campuses, malls, airports, and outdoor locations for convenient navigation. 🚀

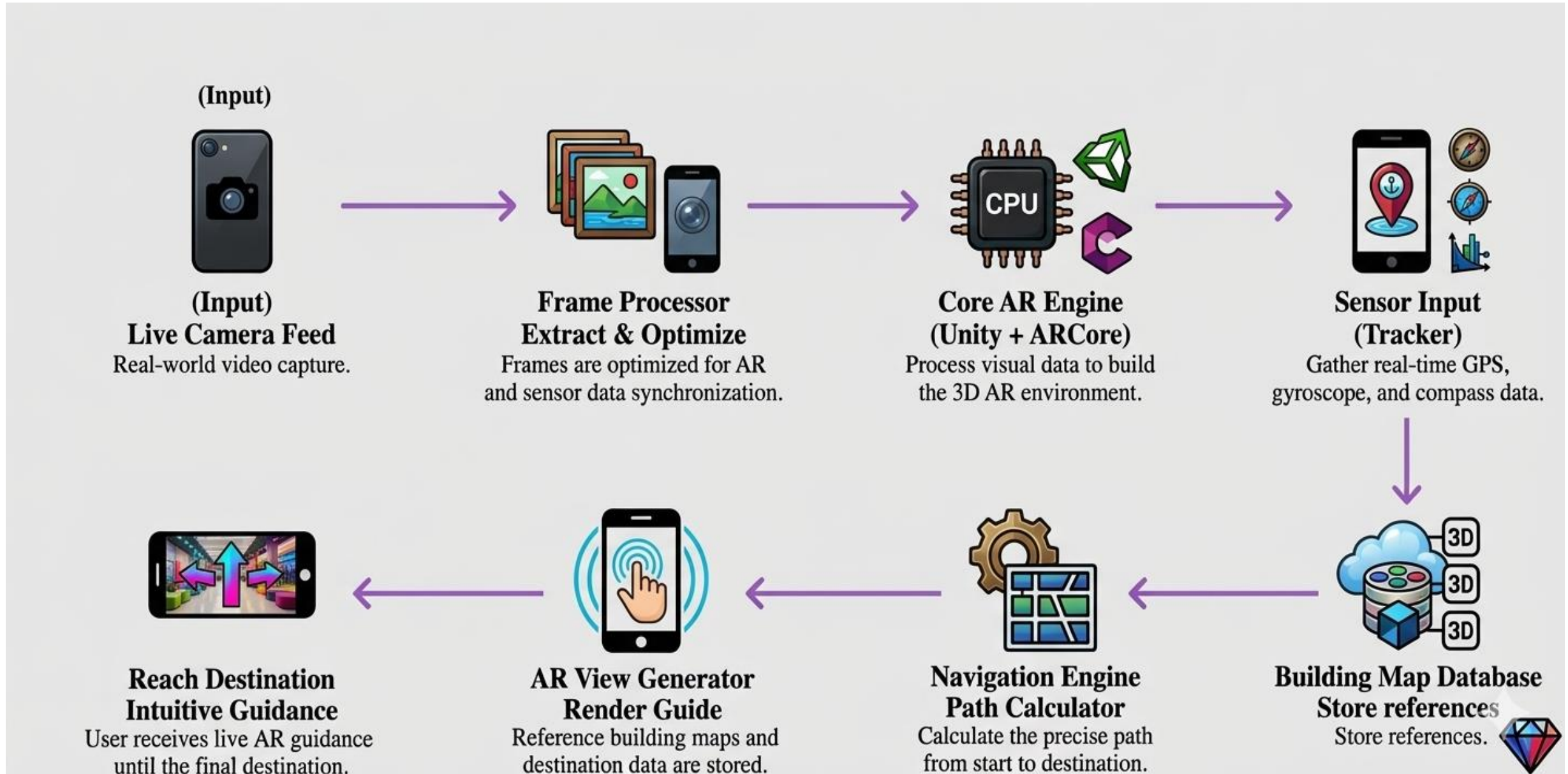
Hardware Requirements

- **Processor & Memory:** Intel Core i5 or higher, **8 GB RAM minimum** (16 GB recommended), **5 GB storage**
- **Camera & Sensors:** HD camera with **GPS, gyroscope, accelerometer, and compass** for location and direction tracking
- **Device & Connectivity:** Smartphone/Tablet/AR glasses with **GPU support and internet connection** for real-time updates

Software Requirements

- **Operating System:** Android / iOS / Windows
- **Development & AR Tools:** Unity 3D or Unreal Engine with AR Core / ARKit
- **Technologies & Services:** C# / C++, Google Maps API / Map box, optional Firebase/MySQL database and Git

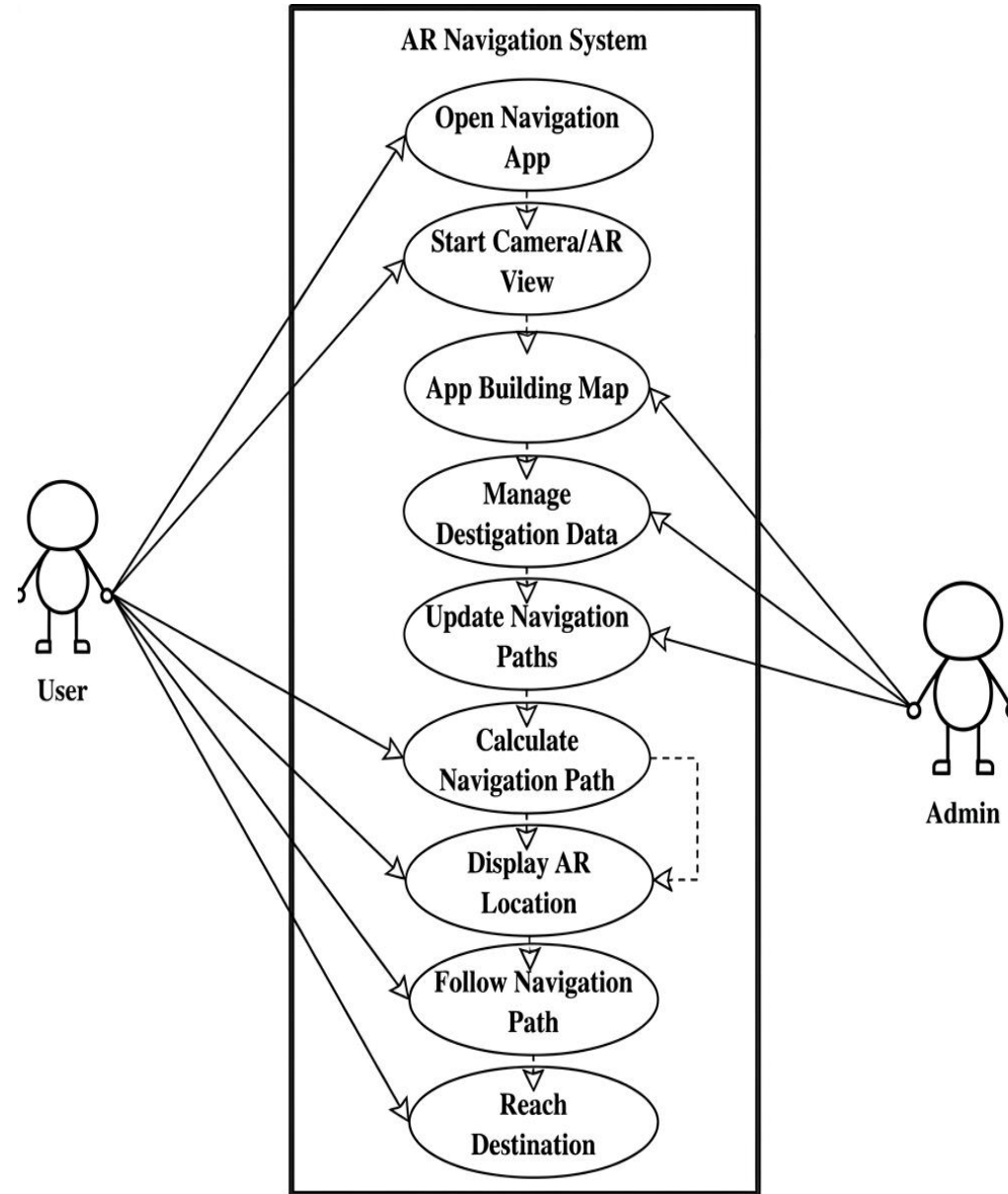
SYSTEM ARCHITECTURE



SYSTEM ARCHITECTURE

- 1. User Device:** Smartphone or AR glasses with **camera and sensors (GPS, accelerometer, gyroscope)** to capture the environment and track user movement.
- 2. Data Layer:** Provides **maps, location data, and real-time traffic/environment updates** from map services.
- 3. Processing Layer:** Uses **computer vision, AR engine, and pathfinding algorithms** to detect surroundings and generate the optimal navigation route.
- 4. Cloud / Backend:** Stores **large map datasets and traffic information** and provides synchronization and updates.
- 5. User Interface:** Displays **AR navigation overlays (arrows, distance markers, directions)** and **voice guidance** for easy navigation.

USE CASE DIAGRAM



USE CASE DIAGRAM

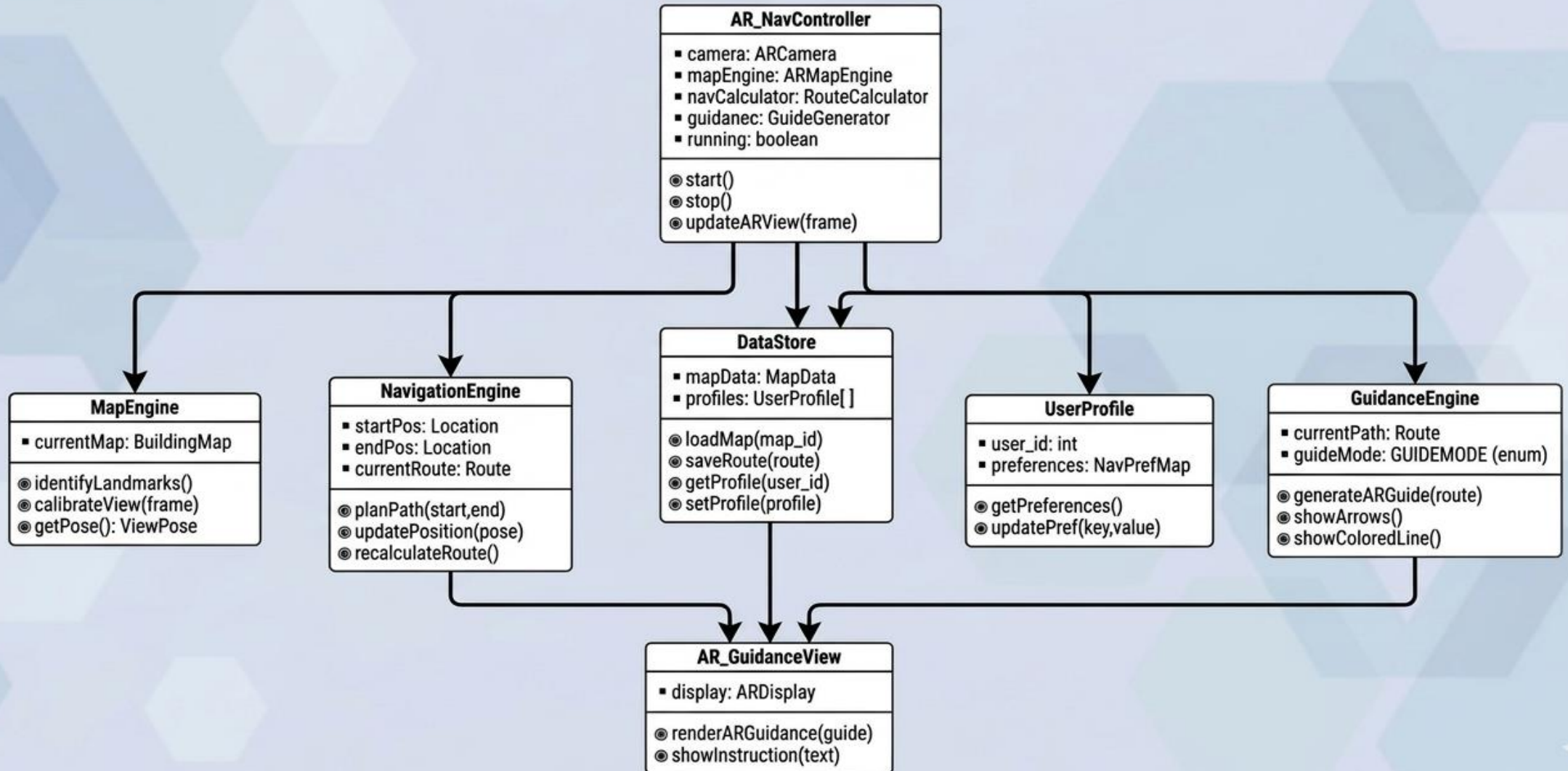
Admin Functions

- Manage Users
- Update Map & POI (Points of Interest) Database
- Monitor System Performance
- Configure System Settings
- Generate Reports

User Functions

- View AR Map with overlays
- Share Location with others
- Access Navigation History
- Explore POIs (restaurants, landmarks, etc.)
- Get Real-time Navigation & Search Destination

CLASS DIAGRAM



CLASS DIAGRAM

1. Core Navigation Components:

Navigation System, Location Tracker, Route Engine, and Map Data work together to manage user location, calculate optimal routes, and provide map information.

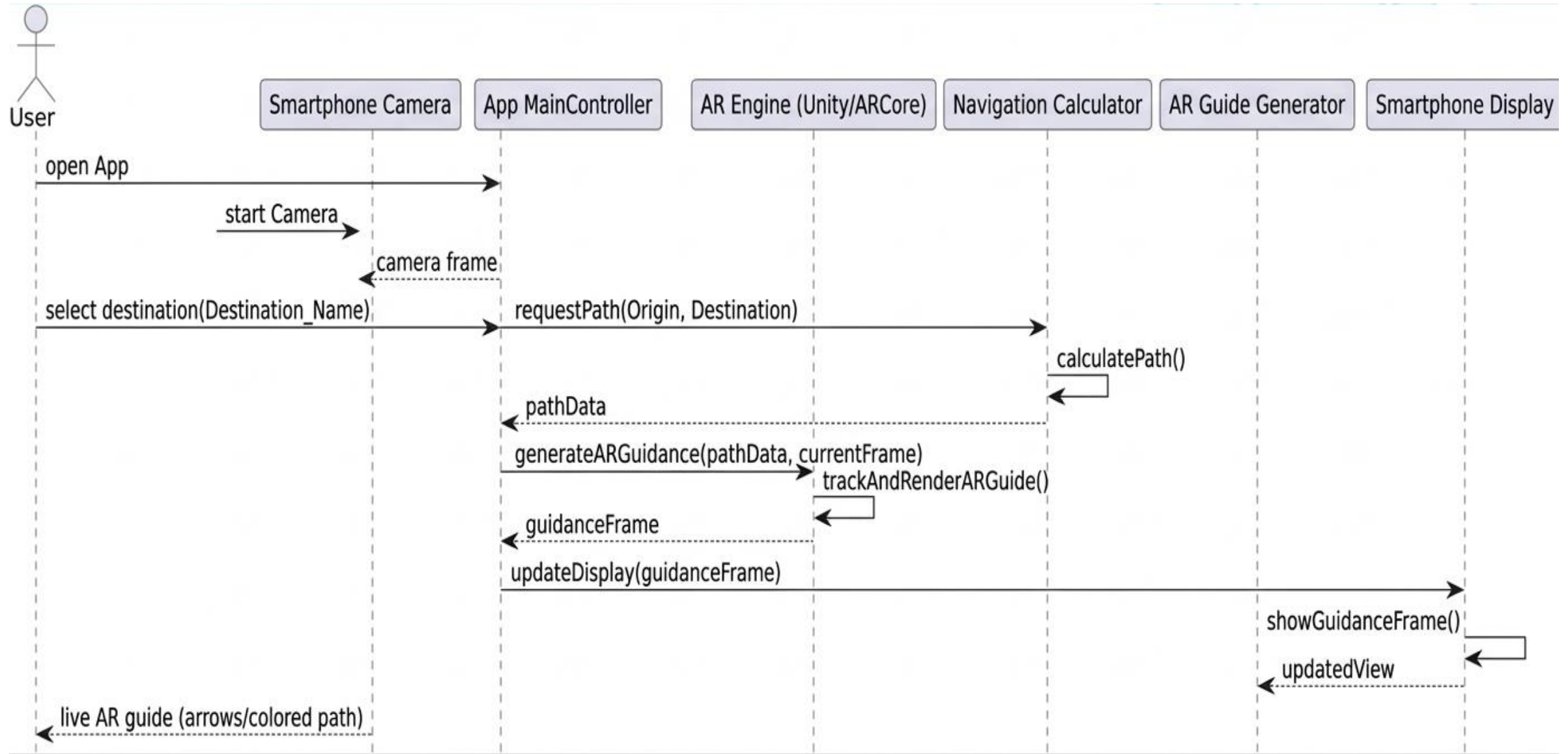
2. AR Visualization & Interaction:

AR Renderer, Camera Stream, AR Elements, and Voice Guidance display navigation directions and points of interest on the live camera view with audio instructions.

3. Supporting Navigation Data:

Location, Waypoints, and Points of Interest (POIs) provide position details, step-by-step navigation, and nearby landmarks or services.

SEQUENCE DIAGRAM



SEQUENCE DIAGRAM

- App Initialization:**

User opens the AR navigation app, which collects real-time **GPS location and device orientation** using sensors like the compass.

- Route Calculation:**

The **Navigation Engine** queries the **map database** and calculates the **optimal route** to the destination.

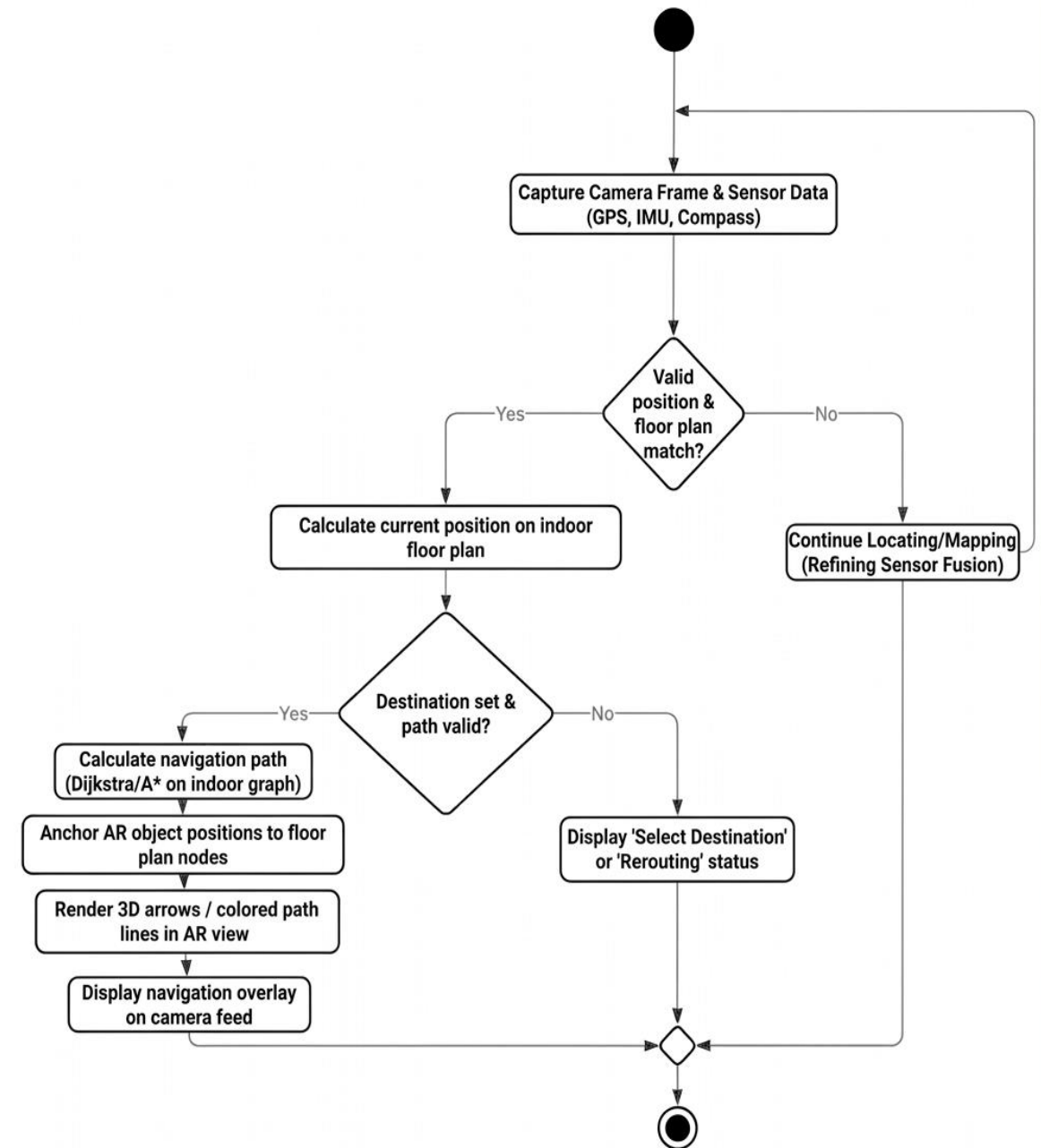
- AR Rendering:**

The **camera provides a live video feed**, and the **rendering engine generates AR markers** such as arrows and directions.

- Real-Time Navigation:**

The system **overlays navigation cues on the live camera view** and continuously updates location and direction until the destination is reached.

ACTIVITY DIAGRAM



ACTIVITY DIAGRAM

- The system **starts by initializing the AR camera** and loading the user's current location.
- The user **selects and confirms the destination** before navigation begins.
- Once confirmed, the system **calculates the best route and retrieves map data**.
- AR markers and navigation instructions are **overlaid on the live camera feed**.
- The system continuously checks if the **user has reached the destination**.
- If the user is still moving, the system **updates position and recalculates the route if needed**.
- If obstacles are detected, the system **alerts the user and suggests an alternative route** until the destination is reached.

MODULE DESCRIPTION

1. Location & Sensor Module:

Collects **real-time user location and device orientation** for accurate tracking.

Technologies: GPS, Compass Sensor, Gyroscope

2. Route Planning Module:

Calculates the **shortest and optimal path** to the destination using map data.

Technologies: Navigation Engine, Map Database, Routing Algorithms

3. Camera & AR Visualization Module:

Captures **live camera feed** and overlays **AR navigation elements** like arrows and markers.

Technologies: AR Core / ARKit, OpenGL, Rendering Engine, Computer Vision

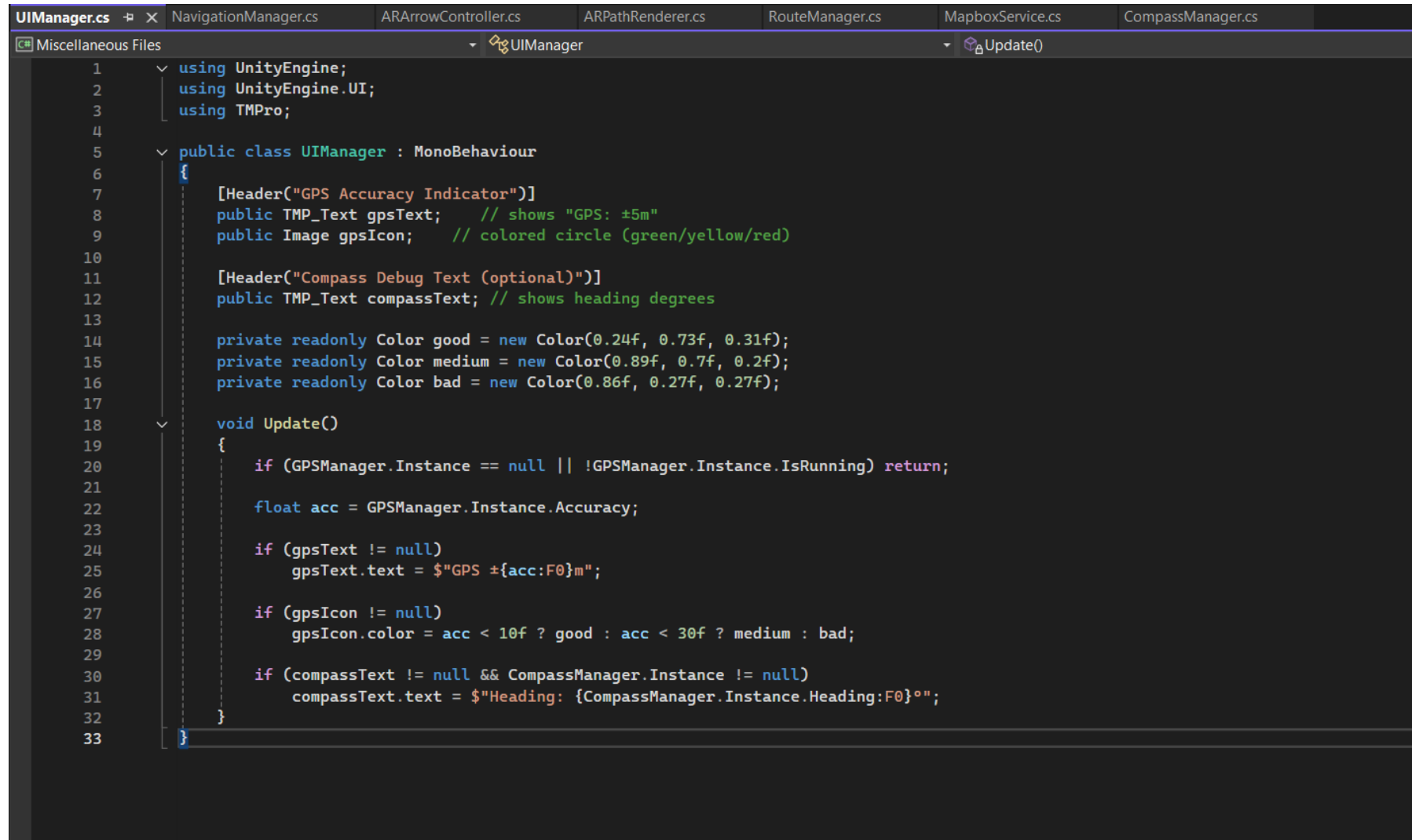
4. Navigation Control & UI Module:

Manages the **navigation process, displays directions, and updates user movement until destination.**

Technologies: Mobile App Framework (Android / iOS), UI Components

IMPLEMENTATION

UIManagers



The screenshot shows a code editor with several tabs at the top: UIManager.cs, NavigationManager.cs, ARArrowController.cs, ARPathRenderer.cs, RouteManager.cs, MapboxService.cs, and CompassManager.cs. The UIManager.cs tab is active, showing the following code:

```
1  using UnityEngine;
2  using UnityEngine.UI;
3  using TMPro;
4
5  public class UIManager : MonoBehaviour
6  {
7      [Header("GPS Accuracy Indicator")]
8      public TMP_Text gpsText;    // shows "GPS: ±5m"
9      public Image gpsIcon;      // colored circle (green/yellow/red)
10
11     [Header("Compass Debug Text (optional)")]
12     public TMP_Text compassText; // shows heading degrees
13
14     private readonly Color good = new Color(0.24f, 0.73f, 0.31f);
15     private readonly Color medium = new Color(0.89f, 0.7f, 0.2f);
16     private readonly Color bad = new Color(0.86f, 0.27f, 0.27f);
17
18     void Update()
19     {
20         if (GPSManager.Instance == null || !GPSManager.Instance.IsRunning) return;
21
22         float acc = GPSManager.Instance.Accuracy;
23
24         if (gpsText != null)
25             gpsText.text = $"GPS ±{acc:F0}m";
26
27         if (gpsIcon != null)
28             gpsIcon.color = acc < 10f ? good : acc < 30f ? medium : bad;
29
30         if (compassText != null && CompassManager.Instance != null)
31             compassText.text = $"Heading: {CompassManager.Instance Heading:F0}°";
32     }
33 }
```

Navigation Manager

```
NavigationManager.cs  ARArrowController.cs  ARPa
NavigationManager
using System.Collections;
using UnityEngine;
using UnityEngine.UI;
using TMPPro;

public class NavigationManager : MonoBehaviour
{
    public static NavigationManager Instance;

    [Header("Panels")]
    public GameObject searchPanel;
    public GameObject navPanel;
    public GameObject loadingPanel;

    [Header("Search Panel UI")]
    public TMP_InputField destinationInput;
    public Button searchBtn;
    public Button startBtn;
    public TMP_Text searchResultText;

    [Header("Navigation Panel UI")]
    public TMP_Text instructionText;
    public TMP_Text distanceText;
    public TMP_Text etaText;
    public Button stopBtn;

    [Header("Status Text")]
    public TMP_Text statusText;

    [Header("AR Components")]
    public ARPathRenderer pathRenderer;
    public ARArrowController arrowController;

    [Header("Transport Mode Buttons")]
    public Button walkBtn;
    public Button bikeBtn;
    public Button carBtn;
```

```
NavigationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs  Ma
NavigationManager
public Button carBtn;

private Color activeColor = new Color(0.1f, 0.7f, 0.3f, 1f);
private Color inactiveColor = new Color(0.2f, 0.2f, 0.2f, 0.8f);

private float destLat, destLon;
private bool hasDestination = false;
private string selectedMode = "walking";

// Store latest GPS for Update loop
private float latestLat = 0f;
private float latestLon = 0f;

private void Awake()
{
    if (Instance == null) Instance = this;
    else Destroy(gameObject);
}

private void Start()
{
    GPSManager.Instance.OnLocationUpdated += OnGPSUpdated;
    RouteManager.Instance.OnStepChanged += OnStepChanged;
    RouteManager.Instance.OnDistanceUpdated += OnDistanceUpdated;
    RouteManager.Instance.OnArrived += OnArrived;

    searchBtn.onClick.AddListener(OnSearchClicked);
    startBtn.onClick.AddListener(OnStartClicked);
    stopBtn.onClick.AddListener(OnStopClicked);
    startBtn.interactable = false;

    if (walkBtn != null) walkBtn.onClick.AddListener(() => SetMode("walking"));
    if (bikeBtn != null) bikeBtn.onClick.AddListener(() => SetMode("cycling"));
    if (carBtn != null) carBtn.onClick.AddListener(() => SetMode("driving"));

    SetMode("walking");
    ShowSearchPanel();
    SetStatus("Starting GPS...");
    StartCoroutine(StartupSequence());
}
```

NavigationManager.cs ARArrowController.cs ARPathRenderer.cs RouteManager.cs MapboxService

NavigationManager

```

}

// — Update — keeps arrow position correct every frame —
private void Update()
{
    if (RouteManager.Instance.IsNavigating &&
        latestLat != 0f && arrowController != null)
    {
        arrowController.UpdateArrowPosition(
            latestLat, latestLon,
            CompassManager.Instance Heading);
    }
}

// — Transport mode —
private void SetMode(string mode)
{
    selectedMode = mode;
    MapboxService.Instance.travelMode = mode;

    if (walkBtn != null) walkBtn.GetComponent<Image>().color =
        mode == "walking" ? activeColor : inactiveColor;
    if (bikeBtn != null) bikeBtn.GetComponent<Image>().color =
        mode == "cycling" ? activeColor : inactiveColor;
    if (carBtn != null) carBtn.GetComponent<Image>().color =
        mode == "driving" ? activeColor : inactiveColor;

    SetBtnText(walkBtn, mode == "walking");
    SetBtnText(bikeBtn, mode == "cycling");
    SetBtnText(carBtn, mode == "driving");
}

private void SetBtnText(Button btn, bool active)
{
    if (btn == null) return;
    var t = btn.GetComponentInChildren<TMP_Text>();
    if (t != null) t.color = active ? Color.white : new Color(0.6f, 0.6f, 0.6f);
}

```

NavigationManager.cs ARArrowController.cs ARPathRenderer.cs RouteManager.cs MapboxService

NavigationManager

```

if (t != null) t.color = active ? Color.white : new Color(0.6f, 0.6f, 0.6f);
}

// — Startup —
private IEnumerator StartupSequence()
{
    StartCoroutine(GPSManager.Instance.StartGPS());
    yield return new WaitForSeconds(1f);
    CompassManager.Instance.EnableCompass();

    float waited = 0f;
    while (!GPSManager.Instance.IsRunning && waited < 30f)
    {
        waited += 1f;
        SetStatus($"Waiting for GPS... ({(int)(30f - waited)}s)");
        yield return new WaitForSeconds(1f);
    }

    SetStatus(GPSManager.Instance.IsRunning
        ? "Ready. Enter destination."
        : "GPS weak. Go outside & retry.");
}

// — Search —
private void OnSearchClicked()
{
    string q = destinationInput.text.Trim();
    if (q.Length == 0) return;

    loadingPanel.SetActive(true);
    SetStatus("Searching...");

    StartCoroutine(MapboxService.Instance.SearchPlace(q,
        (lat, lon, name) =>
        {
            loadingPanel.SetActive(false);
            destLat = lat; destLon = lon;
            hasDestination = true;
        }
    ));
}

```

```
NavigationManager.cs ARArrowController.cs ARPathRenderer.cs RouteManager.cs
NavigationManager
searchResultText.text = "Found: " + name;
setStatus("Tap Start to navigate.");
},
err =>
{
    loadingPanel.SetActive(false);
    searchResultText.text = "Not found.";
    setStatus("Error: " + err);
}
});

// — Start —
private void OnStartClicked()
{
    if (!hasDestination) return;

    if (GPSManager.Instance.Latitude == 0f &&
        GPSManager.Instance.Longitude == 0f)
    { setStatus("No GPS signal. Go outside."); return; }

    loadingPanel.SetActive(true);
    setStatus($"Getting {selectedMode} route...");

    StartCoroutine(MapboxService.Instance.GetRoute(
        GPSManager.Instance.Latitude,
        GPSManager.Instance.Longitude,
        destLat, destLon,
        route =>
        {
            loadingPanel.SetActive(false);
            if (route.waypoints.Count == 0)
            { setStatus("No route found."); return; }

            pathRenderer.SetOrigin(
                GPSManager.Instance.Latitude,
                GPSManager.Instance.Longitude);

            RouteManager.Instance.StartNavigation(route);
        }
    ));
}
```

```
NavigationManager.cs ARArrowController.cs ARPathRenderer.cs RouteManager.cs
NavigationManager
ShowNavPanel();
setStatus("Navigation active!");
},
err =>
{
    loadingPanel.SetActive(false);
    setStatus("Route error: " + err);
}
});

// — Stop —
private void OnStopClicked()
{
    RouteManager.Instance.StopNavigation();
    pathRenderer.ClearPath();
    arrowController.HideArrow();
    ShowSearchPanel();
    setStatus("Stopped.");
}

// — GPS update —
private void OnGPSUpdated(float lat, float lon)
{
    // Store latest GPS for Update loop
    latestLat = lat;
    latestLon = lon;

    RouteManager.Instance.OnGPSUpdate(lat, lon);

    if (RouteManager.Instance.IsNavigating)
    {
        // Update green road line
        pathRenderer.UpdatePath(
            RouteManager.Instance.GetUpcomingWaypoints(30),
            lat, lon,
            CompassManager.Instance.Heading);
    }
}
```


NavigationManager.cs ARArrowController.cs ARPathRenderer.cs RouteManager.cs

NavigationManager

```

CompassManager.Instance.Heading);

// Trigger arrow check
arrowController.ShowArrow(
    RouteManager.Instance.GetCurrentStep());
}

// — Route events —
private void OnStepChanged(RouteStep step)
{
    if (step == null) return;
    instructionText.text = step.instruction;
    float d = step.distance;
    distanceText.text = d >= 1000f
        ? $"{d / 1000f:F1} km"
        : $"{Mathf.RoundToInt(d)} m";
}

private void OnDistanceUpdated(float metres)
{
    float speed = selectedMode == "driving" ? 800f :
        selectedMode == "cycling" ? 250f : 80f;
    etaText.text = $"ETA: ~{Mathf.CeilToInt(metres / speed)} min";
}

private void OnArrived()
{
    instructionText.text = "You have arrived!";
    distanceText.text = "0 m";
    etaText.text = "Arrived";
    arrowController.HideArrow();
    pathRenderer.ClearPath();
}

// — UI helpers —
private void ShowSearchPanel()
{
    searchPanel.SetActive(true);

```

NavigationManager.cs ARArrowController.cs ARPathRenderer.cs

NavigationManager

```

etaText.text = "Arrived";
arrowController.HideArrow();
pathRenderer.ClearPath();
}

// — UI helpers —
private void ShowSearchPanel()
{
    searchPanel.SetActive(true);
    navPanel.SetActive(false);
}

private void ShowNavPanel()
{
    searchPanel.SetActive(false);
    navPanel.SetActive(true);
}

private void SetStatus(string msg)
{
    if (statusText) statusText.text = msg;
    Debug.Log("[Nav] " + msg);
}
}

```

ARArrowController

```
NavigationManager.cs ARArrowController.cs ARPathRenderer.cs RouteManager.cs
Files ARArrowController

using UnityEngine;

// =====
// ARArrowController - ARROW POINTS IN CORRECT DIRECTION
//
// Uses SAME headingOffset as ARPathRenderer so both path
// and arrow always agree with each other.
// =====

public class ARArrowController : MonoBehaviour
{
    public static ARArrowController Instance;

    [Header("Arrow Prefabs - assign in Inspector")]
    public GameObject arrowStraight;
    public GameObject arrowLeft;
    public GameObject arrowRight;
    public GameObject arrowSlightLeft;
    public GameObject arrowSlightRight;
    public GameObject arrowUTurn;
    public GameObject arrowArrived;

    [Header("Placement Settings")]
    public float sideOffset = 1.5f;
    public float heightOffset = 1.0f;
    public float showDistance = 60f;
    public float arrowScale = 2.0f;
    public float straightArrowDistance = 8f;
    public float forwardTilt = -15f;

    [Header("Heading Correction (must match ARPathRenderer)")]
    [Tooltip("Must be same value as ARPathRenderer headingOffset")]
    public float headingOffset = 0f;

    private GameObject activeArrow;
    private string lastKey = "";
    private float turnLat = 0f;
    private float turnLon = 0f;
    private bool hasTurnPoint = false;
    private bool isStraight = false;
}
```

```
NavigationManager.cs ARArrowController.cs ARPathRenderer.cs RouteManager.cs Mapbox
ARArrowController

private string currentManeuver = "";
private string currentModifier = "";

private void Awake()
{
    if (Instance == null) Instance = this;
    else { Destroy(gameObject); return; }
}

public void ShowArrow(RouteStep step)
{
    if (step == null) { HideArrow(); return; }

    Debug.Log($"[Arrow] maneuver='{step.maneuver}' modifier='{step.modifier}'");

    bool isTurn = IsTurnStep(step.maneuver);
    bool isArrive = step.maneuver == "arrive";

    string key = step.maneuver + "_" + step.modifier
                + "_" + step.stepLat + "_" + step.stepLon;
    if (key == lastKey) return;
    lastKey = key;

    currentManeuver = step.maneuver;
    currentModifier = step.modifier ?? "";

    if (activeArrow != null) { Destroy(activeArrow); activeArrow = null; }

    if (isTurn || isArrive)
    {
        isStraight = false;
        turnLat = step.stepLat;
        turnLon = step.stepLon;
        hasTurnPoint = true;

        GameObject prefab = PickPrefab(step.maneuver, step.modifier);
        if (prefab == null) return;

        activeArrow = Instantiate(prefab, Vector3.zero, Quaternion.identity);
    }
}
```

```

NavigationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs  MapboxService.cs
ARArrowController
UpdateArrowPosition

    activeArrow.transform.localScale = Vector3.one * arrowScale;
    activeArrow.SetActive(false);
}
else
{
    isStraight = true;
    hasTurnPoint = false;
    if (arrowStraight == null) return;

    activeArrow = Instantiate(arrowStraight, Vector3.zero, Quaternion.identity);
    activeArrow.transform.localScale = Vector3.one * arrowScale;
    activeArrow.SetActive(true);
}
}

// Called every frame from NavigationManager.Update()
public void UpdateArrowPosition(float userLat, float userLon, float heading)
{
    if (activeArrow == null || Camera.main == null) return;

    // Apply same correction as ARPathRenderer
    float correctedHeading = (heading + headingOffset + 360f) % 360f;

    Transform cam = Camera.main.transform;
    float groundY = cam.position.y - 1.4f + heightOffset;

    if (isStraight)
    {
        // Straight arrow: floats ahead-right of user
        float forwardRad = correctedHeading * Mathf.Deg2Rad;
        float rightRad = (correctedHeading + 90f) * Mathf.Deg2Rad;

        Vector3 arrowPos = new Vector3(
            cam.position.x
            + Mathf.Sin(forwardRad) * straightArrowDistance
            + Mathf.Sin(rightRad) * sideOffset,
            groundY,
            cam.position.z
            + Mathf.Cos(forwardRad) * straightArrowDistance

```

```

NavigationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs  MapboxService.cs
ARArrowController
UpdateArrowPosition

    case "left": return -90f;
    case "slight left": return -45f;
    default: return 0f;
}

private bool IsTurnStep(string maneuver)
{
    if (string.IsNullOrEmpty(maneuver)) return false;
    switch (maneuver)
    {
        case "turn":
        case "fork":
        case "off ramp":
        case "on ramp":
        case "roundabout":
        case "rotary":
        case "roundabout turn":
        case "exit roundabout":
            return true;
        default:
            return false;
    }
}

private GameObject PickPrefab(string maneuver, string modifier)
{
    if (maneuver == "arrive") return arrowArrived;
    if (string.IsNullOrEmpty(modifier)) return arrowStraight;

    switch (modifier.ToLower().Trim())
    {
        case "left":
        case "sharp left": return arrowLeft;
        case "slight left": return arrowSlightLeft != null ? arrowSlightLeft : arrowLeft;
        case "right":
        case "sharp right": return arrowRight;
        case "slight right": return arrowSlightRight != null ? arrowSlightRight : arrowRight;
        case "uturn": return arrowUTurn;

```

NavigationManager.csARArrowController.csARPathRenderer.csRouteManager.csMapboxService.cs

FilesARArrowControllerUpdateArrowPos

```
        case "sharp right": return arrowRight;
        case "slight right": return arrowSlightRight != null ? arrowSlightRight : arrowRight;
        case "uturn": return arrowUTurn;
        case "straight": return arrowStraight;
        default:
            Debug.LogWarning($"[Arrow] Unknown modifier '{modifier}'");
            return arrowStraight;
    }
}

public void HideArrow()
{
    if (activeArrow != null) { Destroy(activeArrow); activeArrow = null; }
    hasTurnPoint = false;
    isStraight = false;
    lastKey = "";
}
```

ARPathRenderer

```
NavigationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs
Files
using System.Collections.Generic;
using UnityEngine;

// =====
// ARPathRenderer - GREEN ROAD LINE
//
// Converts each Mapbox GPS waypoint into AR world position
// using bearing + distance from user position.
// HeadingOffset corrects compass errors.
// =====
[RequireComponent(typeof(LineRenderer))]
public class ARPathRenderer : MonoBehaviour
{
    public static ARPathRenderer Instance;

    [Header("Line Settings")]
    public float lineWidth = 0.4f;
    public float lineHeight = 0.15f;
    public float maxDistance = 80f;
    public int maxPoints = 30;

    [Header("Heading Correction")]
    [Tooltip("If path points wrong direction, adjust this. Try +90 or -90")]
    public float headingOffset = 0f;

    private LineRenderer lr;

    private void Awake()
    {
        if (Instance == null) Instance = this;
        else { Destroy(gameObject); return; }
        lr = GetComponent<LineRenderer>();
        SetupLine();
    }

    private void SetupLine()
    {
        lr.startWidth = lineWidth;
        lr.endWidth = lineWidth;
```

```
NavigationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs
Files
    lr.useWorldSpace = true;
    lr.numCornerVertices = 6;
    lr.numCapVertices = 6;
    lr.shadowCastingMode = UnityEngine.Rendering.ShadowCastingMode.Off;
    lr.receiveShadows = false;
    lr.positionCount = 0;

    var mat = new Material(Shader.Find("Unlit/Color"));
    mat.color = Color.white;
    lr.material = mat;

    // Force green via gradient - prevents pink color space issue
    lr.colorGradient = MakeGradient(Color.green);
}

private Gradient MakeGradient(Color c)
{
    var g = new Gradient();
    g.SetKeys(
        new[] { new GradientColorKey(c, 0f), new GradientColorKey(c, 1f) },
        new[] { new GradientAlphaKey(1f, 0f), new GradientAlphaKey(1f, 1f) }
    );
    return g;
}

public void SetOrigin(float lat, float lon) { }

// =====
// UpdatePath - called every GPS update
//
// For each waypoint:
//   bearing   = compass direction from user to waypoint
//   distance  = metres to waypoint
//   relAngle  = bearing - phone heading (corrected)
//   X         = sin(relAngle) * distance
//   Z         = cos(relAngle) * distance
// =====
```

```
vigationManager.cs  ARArrowController.cs  ARPathRender.cs  RouteManager.cs  Map
ARPathRender.cs
{ ClearPath(); return; }

Transform cam = Camera.main.transform;
float groundY = cam.position.y - 1.4f + lineHeight;
var pts = new List<Vector3>();

// Apply heading correction
float correctedHeading = (heading + headingOffset + 360f) % 360f;

// Start at user feet
pts.Add(new Vector3(cam.position.x, groundY, cam.position.z));

foreach (var wp in waypoints)
{
    float dist = GPSManager.DistanceMeters(userLat, userLon, wp.x, wp.y);
    if (dist < 2f) continue;
    if (dist > maxDistance) break;

    float bearing = CompassManager.BearingTo(userLat, userLon, wp.x, wp.y);
    float relAngle = (bearing - correctedHeading) * Mathf.Deg2Rad;

    pts.Add(new Vector3(
        cam.position.x + Mathf.Sin(relAngle) * dist,
        groundY,
        cam.position.z + Mathf.Cos(relAngle) * dist
    ));

    if (pts.Count >= maxPoints) break;
}

if (pts.Count < 2) { ClearPath(); return; }
lr.positionCount = pts.Count;
lr.SetPositions(pts.ToArray());
}

public void ClearPath()
{
    if (lr != null) lr.positionCount = 0;
}
```


RouteManager

```
NavigationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs  Ma
les
  ▼ RouteManager
  ▼
  using System.Collections.Generic;
  using UnityEngine;

  public class RouteManager : MonoBehaviour
  {
      public static RouteManager Instance;

      // Read-only state
      public MapboxRoute CurrentRoute { get; private set; }
      public int WaypointIndex { get; private set; }
      public int StepIndex { get; private set; }
      public bool IsNavigating { get; private set; }

      [Header("Distance Thresholds")]
      public float waypointReachedMeters = 15f;
      public float stepReachedMeters = 20f;

      // Events - subscribe to these in other scripts
      public System.Action<RouteStep> OnStepChanged;
      public System.Action<float> OnDistanceUpdated; // remaining metres
      public System.Action OnArrived;

      private void Awake()
      {
          if (Instance == null) { Instance = this; DontDestroyOnLoad(gameObject); }
          else Destroy(gameObject);
      }

      public void StartNavigation(MapboxRoute route)
      {
          CurrentRoute = route;
          WaypointIndex = 0;
          StepIndex = 0;
          IsNavigating = true;

          if (route.steps.Count > 0)
              OnStepChanged?.Invoke(route.steps[0]);

          Debug.Log("Navigation started!");
      }
  }
```

```
ationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs  MapboxService.cs
  ▼ RouteManager
  ▼ waypointRea
  }

  public void StopNavigation()
  {
      IsNavigating = false;
      CurrentRoute = null;
      Debug.Log("Navigation stopped.");
  }

  // Call this every time GPS updates (called from NavigationManager)
  public void OnGPSUpdate(float lat, float lon)
  {
      if (!IsNavigating || CurrentRoute == null) return;

      // Check if we passed the current waypoint
      if (WaypointIndex < CurrentRoute.waypoints.Count)
      {
          var wp = CurrentRoute.waypoints[WaypointIndex];
          float d = GPSManager.DistanceMeters(lat, lon, wp.x, wp.y);
          if (d < waypointReachedMeters) WaypointIndex++;
      }

      // Check if we reached the next step's trigger point
      if (StepIndex < CurrentRoute.steps.Count - 1)
      {
          var nextStep = CurrentRoute.steps[StepIndex + 1];
          float d = GPSManager.DistanceMeters(lat, lon, nextStep.stepLat, nextStep.stepLon);
          if (d < stepReachedMeters)
          {
              StepIndex++;
              OnStepChanged?.Invoke(CurrentRoute.steps[StepIndex]);
          }
      }

      // Calculate remaining distance and check arrival
      if (CurrentRoute.waypoints.Count > 0)
      {
          var dest = CurrentRoute.waypoints[CurrentRoute.waypoints.Count - 1];
          float remaining = GPSManager.DistanceMeters(lat, lon, dest.x, dest.y);
      }
  }
```

```
gationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs  + X M
RouteManager
{
    var dest = CurrentRoute.waypoints[CurrentRoute.waypoints.Count - 1];
    float remaining = GPSManager.DistanceMeters(lat, lon, dest.x, dest.y);
    OnDistanceUpdated?.Invoke(remaining);

    if (remaining < waypointReachedMeters)
    {
        IsNavigating = false;
        OnArrived?.Invoke();
    }
}

// Get the next N waypoints from current position (for the green line)
public List<Vector2> GetUpcomingWaypoints(int count = 25)
{
    if (CurrentRoute == null) return new List<Vector2>();
    int start = Mathf.Clamp(WaypointIndex, 0, CurrentRoute.waypoints.Count);
    int end = Mathf.Min(start + count, CurrentRoute.waypoints.Count);
    if (end <= start) return new List<Vector2>();
    return CurrentRoute.waypoints.GetRange(start, end - start);
}

public RouteStep GetCurrentStep()
{
    if (CurrentRoute == null || CurrentRoute.steps.Count == 0) return null;
    int idx = Mathf.Clamp(StepIndex, 0, CurrentRoute.steps.Count - 1);
    return CurrentRoute.steps[idx];
}
```


MapBoxService

```
NavigationManager.cs ARArrowController.cs ARPathRenderer.cs
Files
  MapboxService
    using System.Collections;
    using System.Collections.Generic;
    using UnityEngine;
    using UnityEngine.Networking;

    [System.Serializable]
    public class RouteStep
    {
        public string instruction;
        public string maneuver;
        public string modifier;
        public float distance;
        public float stepLat;
        public float stepLon;
    }

    [System.Serializable]
    public class MapboxRoute
    {
        public List<Vector2> waypoints = new List<Vector2>();
        public List<RouteStep> steps = new List<RouteStep>();
        public float totalDistance;
        public float totalDuration;
    }

    public class MapboxService : MonoBehaviour
    {
        public static MapboxService Instance;

        [Header("Mapbox Token")]
        public string accessToken = "pk.eyJ1IjoiaWZldmZ2ZWt2aXZlZlE1";

        // Transport mode - set by UI buttons
        // "walking" | "cycling" | "driving"
        [HideInInspector]
        public string travelMode = "walking";

        private void Awake()
        {

```

```
ationManager.cs ARArrowController.cs ARPathRenderer.cs RouteManager.cs MapboxService.cs
  MapboxService
    accessToken

    if (Instance == null) { Instance = this; DontDestroyOnLoad(gameObject); }
    else Destroy(gameObject);
  }

  // — Search place —
  public IEnumerator SearchPlace(string query,
    System.Action<float, float, string> onResult,
    System.Action<string> onError = null)
  {
    string encoded = UnityWebRequest.EscapeURL(query);
    string url = $"https://api.mapbox.com/geocoding/v5/mapbox.places/{encoded}.json"
      + $"?access_token={accessToken}&limit=5";

    using (var www = UnityWebRequest.Get(url))
    {
      yield return www.SendWebRequest();
      if (www.result != UnityWebRequest.Result.Success)
      { onError?.Invoke(www.error); yield break; }

      var root = SimpleJSON.JSON.Parse(www.downloadHandler.text);
      var features = root["features"];
      if (features.Count == 0) { onError?.Invoke("No results found"); yield break; }

      var first = features[0];
      float lon = first["center"][0].AsFloat;
      float lat = first["center"][1].AsFloat;
      string name = first["place_name"].Value;
      onResult?.Invoke(lat, lon, name);
    }
  }

  // — Get route —
  public IEnumerator GetRoute(
    float fromLat, float fromLon,
    float toLat, float toLon,
    System.Action<MapboxRoute> onSuccess,
    System.Action<string> onError = null)
  {
    // Use selected travel mode

```

```

NavigationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs  MapboxService.cs  CompassManager.cs
MapboxService
accessToken

var root = SimpleJSON.JSON.Parse(json);

if (root == null || root["routes"].Count == 0)
{ Debug.LogError("Mapbox: no routes."); return route; }

var r = root["routes"][0];
route.totalDistance = r["distance"].AsFloat;
route.totalDuration = r["duration"].AsFloat;

// All road coordinates
var coords = r["geometry"]["coordinates"];
for (int i = 0; i < coords.Count; i++)
    route.waypoints.Add(new Vector2(coords[i][1].AsFloat, coords[i][0].AsFloat));

// Turn-by-turn steps
var legs = r["legs"];
for (int i = 0; i < legs.Count; i++)
{
    var steps = legs[i]["steps"];
    for (int j = 0; j < steps.Count; j++)
    {
        var s = steps[j];
        route.steps.Add(new RouteStep
        {
            instruction = s["maneuver"]["instruction"].Value,
            maneuver = s["maneuver"]["type"].Value,
            modifier = s["maneuver"]["modifier"].Value,
            distance = s["distance"].AsFloat,
            stepLat = s["maneuver"]["location"][1].AsFloat,
            stepLon = s["maneuver"]["location"][0].AsFloat
        });
    }
}

Debug.Log($"Route: {route.waypoints.Count} points, {route.steps.Count} steps, mode={travelMode}");
return route;
}

```

```

NavigationManager.cs  ARArrowController.cs  ARPathRenderer.cs  RouteManager.cs  MapboxService.cs  CompassManager.cs
CompassManager

using System.Collections;
using UnityEngine;

public class CompassManager : MonoBehaviour
{
    public static CompassManager Instance;

    public float Heading { get; private set; }
    public bool IsReady { get; private set; }

    [Header("Smoothing 0=instant 1=very smooth")]
    public float smoothFactor = 0.2f;

    private float smoothed = 0f;
    private bool running = false;

    private void Awake()
    {
        if (Instance == null) { Instance = this; DontDestroyOnLoad(gameObject); }
        else Destroy(gameObject);
    }

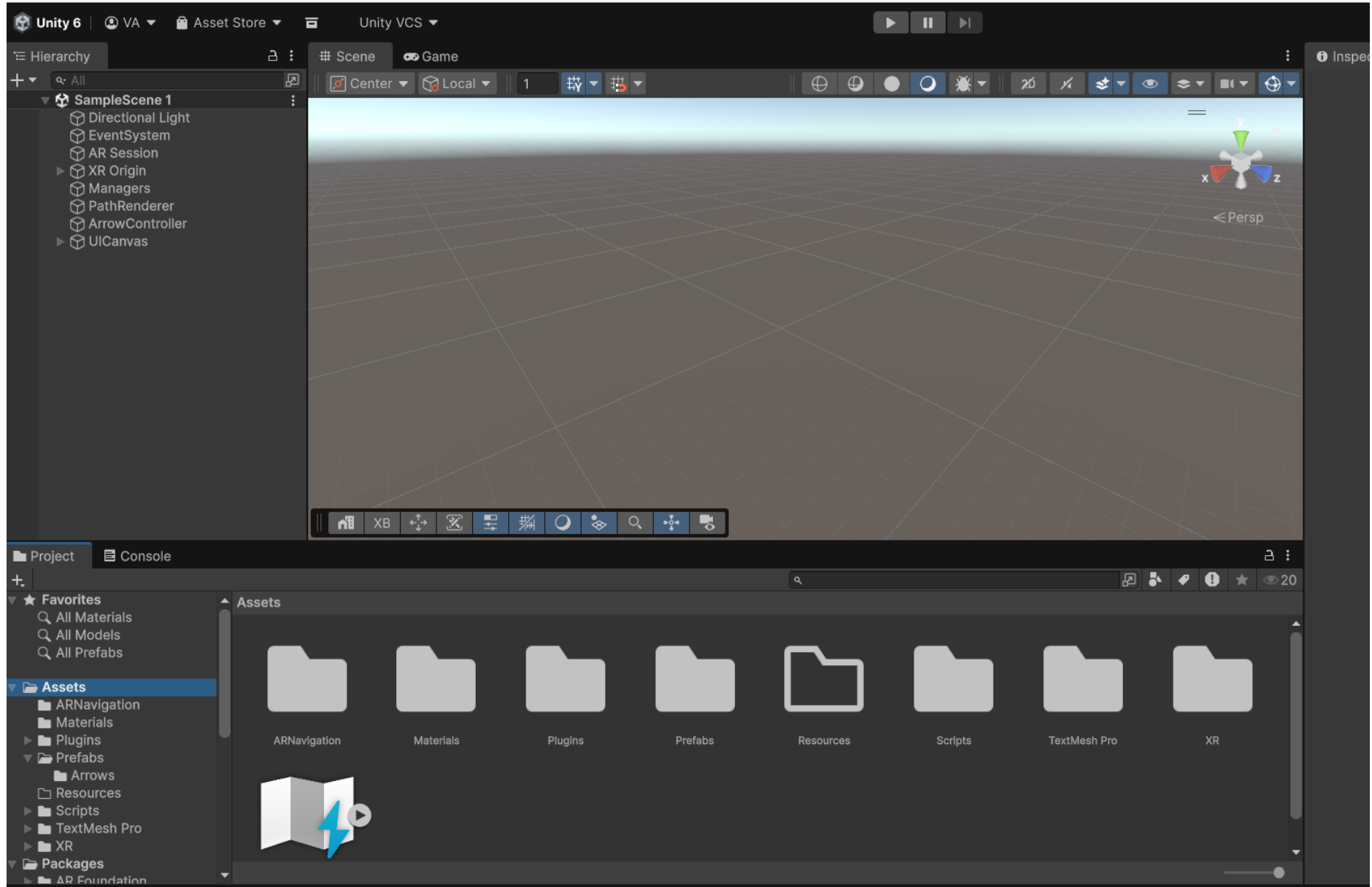
    public void EnableCompass()
    {
        StartCoroutine(Init());
    }

    private IEnumerator Init()
    {
        Input.compass.enabled = true;
        Input.location.Start();

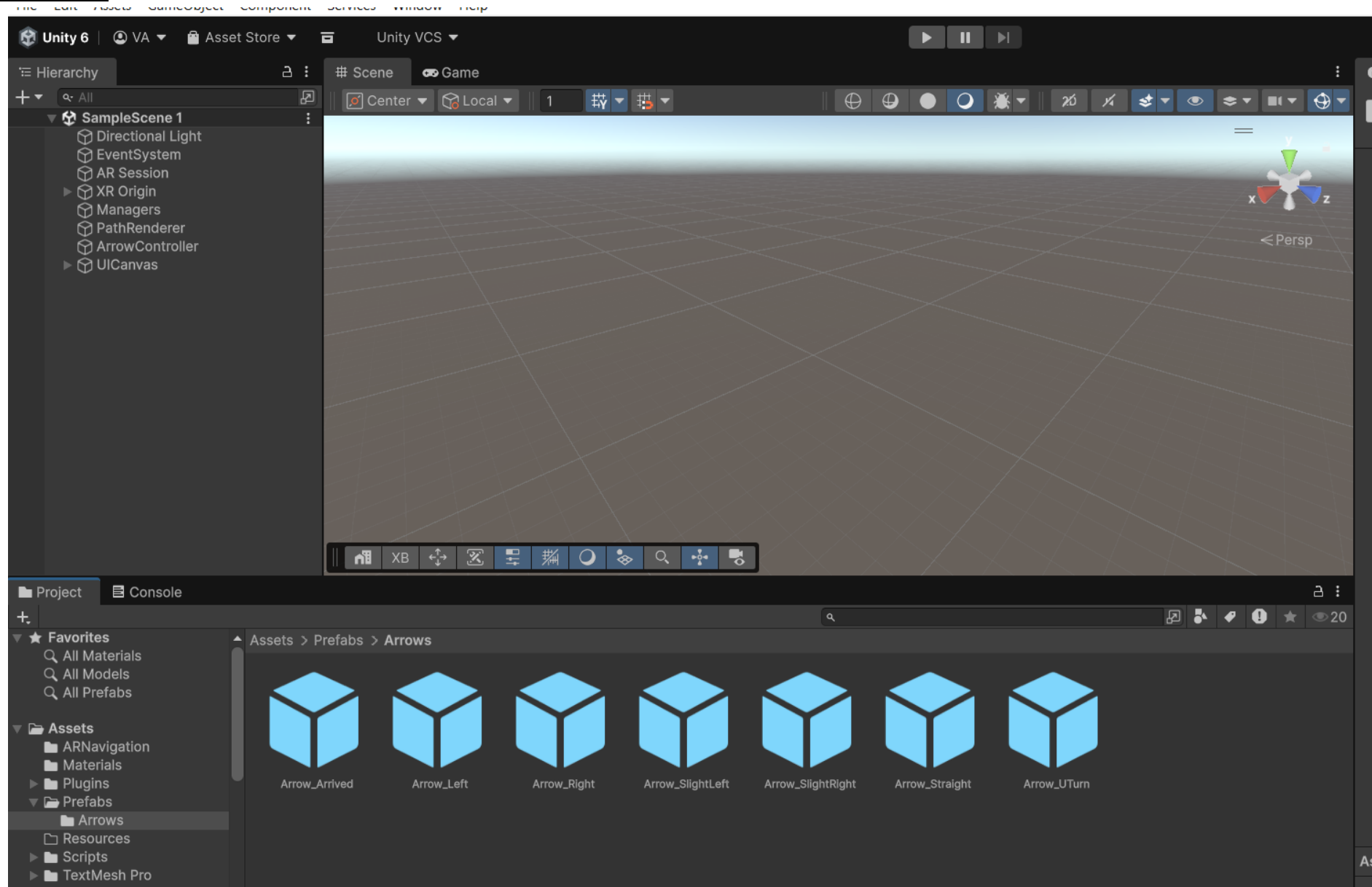
        // Wait for a non-zero heading (up to 15 seconds)
        float timer = 0f;
        while (timer < 15f)
        {
            if (Input.compass.trueHeading != 0f)
            {
                smoothed = Input.compass.trueHeading;
            }
        }
    }
}

```

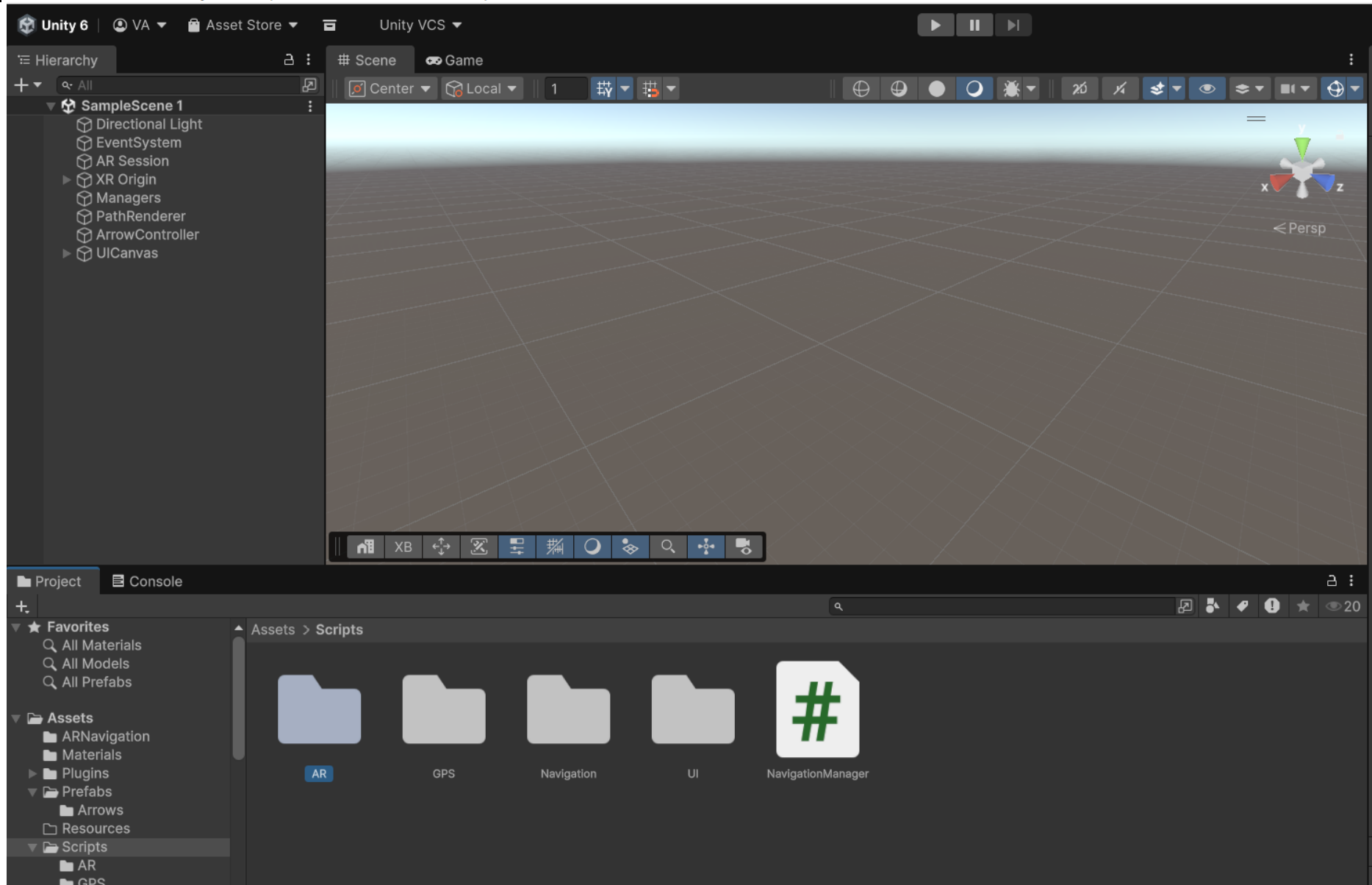
Unity Assets



Pregfabs Arrows



Assets Scripts



OUT PUT

Ready. Enter destination.

☐ Bike ☒ Walk ☐ Car

GPS ±4m
Heading: 0°

AR Navigation

Enter destination

Search

Start Navigation

SearchResultText

Searching...

Loading...

☐ Bike ☒ Walk ☐ Car

GPS: 54m
Heading: 0°

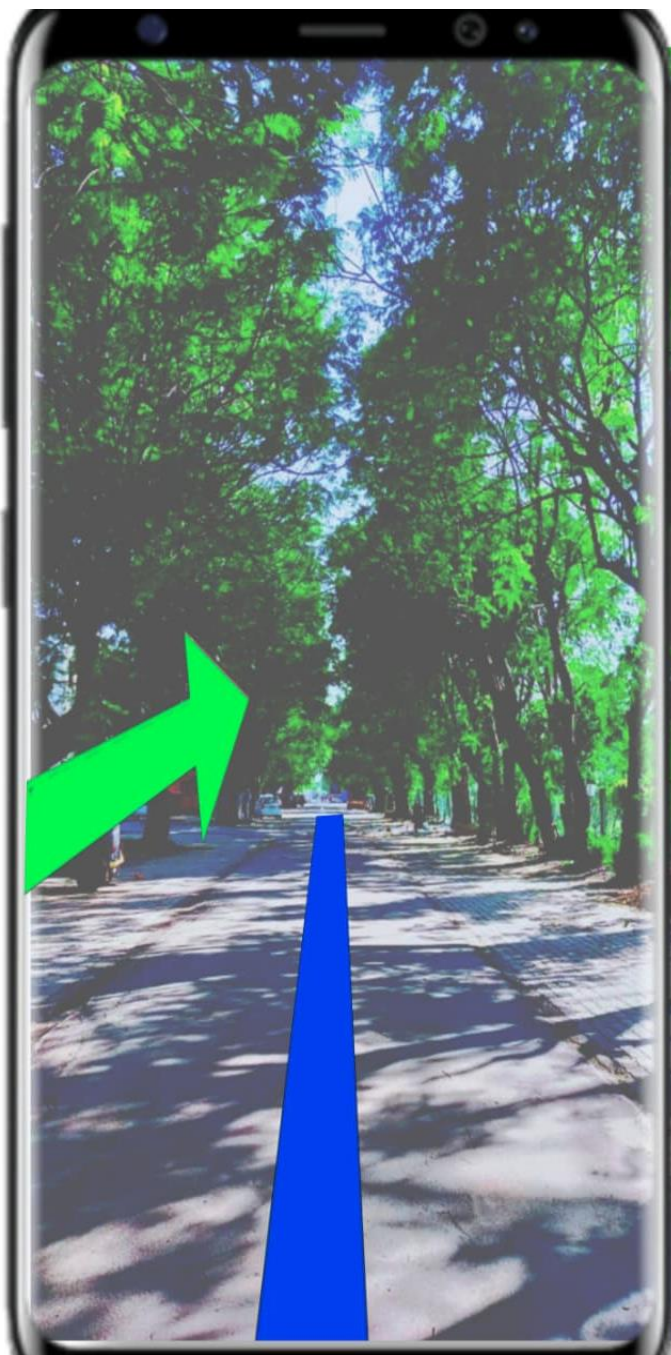
AR Navigation

chammar

Search

Start Navigation

SearchResultText



Thank you

BY TEAM-11

K.Laxman 235U1A6732

G.Srivani 235U1A6721

T.Deekshitha 235U1A6757

J.Madhumitha 235U1A6726